

Operating Systems Report

Document Indexing and Search Service

Group 19

Group members:

- **Name:** José Pedro Flores Novais
Number: a108395
- **Name:** Samir Mansour
Number: a105856
- **Name:** Fábio Mendes Castelhana
Number: a105728

Summary

1. Introduction
2. System Architecture
3. Interprocess Communication
4. Implemented Features
 - 4.1. Document Indexing (-a)
 - 4.2. Metadata Query (-c)
 - 4.3. Document Removal (-d)
 - 4.4. Search Within Document (-l)
 - 4.5. Global Search (-s)
 - 4.6. Server Shutdown (-f)
5. Advanced Features and Optimizations
 - 5.1. Concurrent Search
 - 5.2. Persistence
 - 5.3. Metadata Cache

1. Introduction

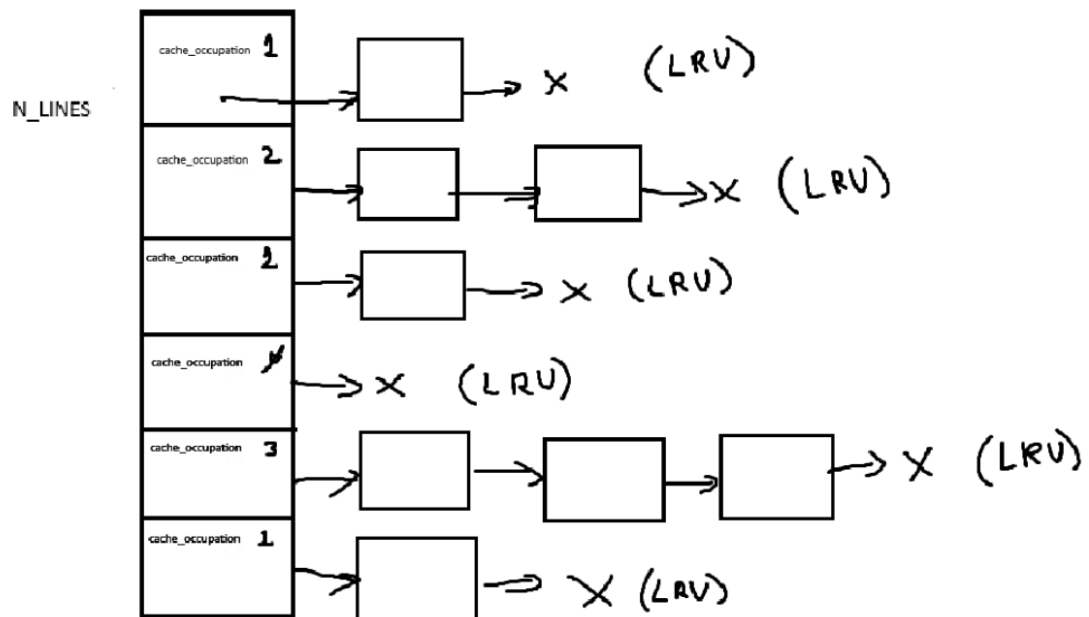
As part of the Operating Systems course at the University of Minho, 2nd year of the Computer Science degree, this report explains the main decisions made regarding process architecture, features, and the specific use of communication mechanisms.

2. System Architecture

Below, the cache and the storage are represented to clarify their structure:

Cache structure

N_LINES: 6
 MAX_CACHE_SIZE: 18
 MAX_LINE_SIZE: 18/6 = 3



fd_file_read



"Storage"

imd Ø	imd 1	imd 1	imd 1
imd 1	imd 1	imd Ø	

fd_file_write

3. Interprocess Communication

Among the processes present in the project, there are different forms of communication between them.

Between the client and the server, the communication mechanism used was FIFO. The server creates a fixed FIFO called `PIPE_TO_SERVER`, and the client always opens this FIFO in write mode and sends a `FileInfo` struct. To receive data from the server, the client creates a temporary FIFO based on its PID (the details of this creation were omitted in previous sections to focus on more relevant technical content). The server opens this FIFO based on the PID received in the `FileInfo` struct and sends a response to the client. The client reads the response and deletes the temporary FIFO.

Between server processes themselves, the communication method used was pipe. In the `-s` command, the goal is for child processes created with `fork()` to inform the parent process about which documents contain the searched word. In the `-l` command, pipes were used to build a pipeline between subprocesses (`grep` and `wc`), where both act as children and the parent receives the numeric result at the end of the pipeline.

4. Implemented Features

Below are the main strategies and decisions taken for each implemented feature.

4.1. Document Indexing (-a)

On the client side, the `FileInfo` struct stores the command-line arguments. These are then written to the FIFO `PIPE_TO_SERVER`.

The server reads the data from `PIPE_TO_SERVER`, copies it into a `file_struct`, and indexes the information into persistent storage. Then, a message with the assigned ID is generated and written to the `fifo_client`.

Finally, the client reads the message from `fifo_client` and prints it to the screen.

4.2. Metadata Query (-c)

The client fills the `FileInfo` struct with the document ID to query. This data is written to `PIPE_TO_SERVER`.

The server reads it and searches for the document using the given ID. It first checks the cache for a faster response. If not found, it searches the storage. If found there, the document is added to the cache for future fast access.

Then, the server prepares a message with the result (or an error message) and writes it to the client FIFO. The client reads and prints this message.

4.3. Document Removal (-d)

The client writes the document ID to be removed into a FileInfo struct and sends it via PIPE_TO_SERVER.

The server reads it, searches the cache first, and removes the entry if found. Regardless of that, it then searches the storage, and if the document is found, sets its active field to 0. This does not delete the file content, only its metadata reference.

A confirmation or error message is then sent to the client FIFO and printed by the client.

4.4. Search Within Document (-l)

The client writes the ID and keyword into a FileInfo struct and sends it via PIPE_TO_SERVER.

The server reads it and calls search_keyword(). If the document is in the cache, grep_wc() is called using the cached path. If it's only in storage, it is read, passed to grep_wc(), and then cached.

grep_wc() creates two pipes:

- pipe1 connects grep to wc,
- pipe2 connects wc to the parent.

Child 1 runs grep, redirecting its output to pipe1.

Child 2 runs wc -l, reading from pipe1 and writing to pipe2.

The parent reads the result from pipe2, formats it, and sends it to the client FIFO.

4.5. Global Search (-s)

The client fills a FileInfo struct with the keyword (and optionally, number of processes), and sends it via PIPE_TO_SERVER.

The server calls search_contains_word(), which first filters out inactive records (if any were removed). It then distributes the search across multiple child processes. Each process searches a range of documents and writes matching IDs into a pipe.

The parent process gathers results from all pipes, composes a formatted list, and sends it to the client FIFO. The client reads and prints it.

4.6. Server Shutdown (-f)

The client sends a FileInfo struct with the -f command via PIPE_TO_SERVER, then immediately closes its FIFO and exits (it doesn't wait for a response).

The server receives the shutdown command, prints a message, and initiates shutdown.

- It compacts storage by rewriting only active documents to a new file (temp) and replaces the original (rename()), thus cleaning up documents removed via -d.

- It frees all cache memory, closes all file descriptors, removes the FIFO, and ends execution with return 0.
-

5. Advanced Features and Optimizations

To improve performance and completeness, the group implemented several optimizations.

5.1. Concurrent Search

In -s, concurrency is achieved via:

- Creating pipes (one per child)
- Dividing the document set into nearly equal parts
- Spawning multiple child processes with fork()

Each child:

- Uses lseek() to reach its range
- Reads documents
- Calls file_contains_word() (which uses execlp with grep -q)
- Sends found IDs to the parent via pipe

The parent:

- Closes write ends of pipes
 - Collects results from children
 - Formats the final message and waits for all children to finish
-

5.2. Persistence

Persistence is implemented via the active flag in indexed_file.

On -f, the server:

- Creates a temp file
- Copies only active documents to it
- Replaces the original using rename("temp", storage_path)

This removes entries marked inactive by -d, effectively **compacting the storage**.

Upon restarting, the server reads from disk and resumes its previous state.

5.3. Metadata Cache

Initially, a singly linked list was considered, but it was replaced by a **hash table with chaining** for better insertion/removal performance.

The hash function is:

$\text{hash} = \text{id} \% \text{N_LINES};$

Where id is the document's unique identifier and N_LINES is the cache size.

The replacement policy used is **Least Recently Used (LRU)** per line. When a line reaches its capacity, the least recently accessed entry is removed:

$\text{virtual_time} - \text{last_accessed}$

Each time a document is accessed or searched, last_accessed is updated to the current virtual_time, maintaining an artificial timeline to support proper LRU behavior.